Predicting music genres from audio data using the Million Songs

Problem Description :

The aim of this project is to predict the genre of a song from its audio data using the Million Songs dataset. The Million Songs dataset is a large, freely-available collection of audio features and metadata for a million contemporary popular music tracks. The dataset includes audio features such as tempo, loudness, and timbre, as well as metadata such as artist, album, and year of release.

Dataset:

The Million Songs dataset is available on Kaggle (https://www.kaggle.com/c/msdchallenge/data). The dataset consists of audio features and metadata for a million contemporary popular music tracks. The audio features include things like tempo, loudness, and timbre, while the metadata includes information about the artist, album, and year of release.

Objectives:

The main objectives of this project are:

- To preprocess the audio data and extract relevant features
- To build a machine learning model to predict the genre of a song based on its audio data
- To evaluate the performance of the model and fine-tune its hyperparameters if necessary

Deliverables:

The deliverables of this project include:

- Preprocessed audio data with relevant features extracted
- Trained machine learning model for genre classification
- Model performance evaluation metrics such as accuracy, precision, and recall
- Fine-tuned model with optimized hyperparameters

Background:

Music genre classification is a challenging task due to the wide range of styles and subgenres that exist within contemporary popular music. In recent years, machine learning algorithms have been applied to this problem with some success. Features such as tempo, loudness, and timbre have been found to be useful in distinguishing between different genres.

The Million Songs dataset provides a rich source of audio features and metadata for a large number of popular music tracks, making it an ideal dataset for training and evaluating machine learning models for music genre classification.

Possible framework:

1. Data Collection

- Collect the Million Songs dataset from Kaggle or any other relevant source.
- Understand the dataset and the different features included in it.

2. Data Preprocessing

- Load the dataset using Pandas or any other relevant library.
- Remove any duplicates, missing values or irrelevant columns.
- Transform the data into a suitable format for machine learning models.

3. Feature Extraction and Engineering

- Explore different audio features such as tempo, spectral centroid, chroma features, etc.
- Extract the relevant audio features using libraries such as Librosa.
- Analyze the correlation between different features and the target variable.
- Engineer new features that may improve the model's performance.

4. Model Building

- Split the dataset into training and testing sets.
- Select appropriate machine learning models such as Random Forest, SVM, or Neural Networks.
- Train the models on the training set.
- Evaluate the models on the testing set using appropriate evaluation metrics such as accuracy, precision, recall, etc.

5. Model Selection and Hyperparameter Tuning

- Compare the performance of different models and select the best performing one.
- Fine-tune the hyperparameters of the selected model using techniques such as Grid Search or Random Search.

6. Model Evaluation

- Evaluate the final model on the testing set using appropriate evaluation metrics.
- Analyze the strengths and weaknesses of the model.
- Visualize the results using appropriate tools such as confusion matrix, ROC curve, etc.

7. Deployment

- Deploy the model in a suitable format such as a web application or a REST API.
- Test the deployed model using new data.

8. Maintenance

- Monitor the model's performance over time.
- Re-train the model on new data if necessary.
- Improve the model's performance by incorporating new features or using advanced techniques.

Code Explanation :

Here is the simple explanation for the code which is provided in the code.py file.

This code is designed to classify the genre of songs using audio features extracted from the Million Song Dataset. The dataset contains various audio features like tempo, beats, timbre, etc., and each song is labeled with one of 16 different genres. The objective of this code is to build a classification model to predict the genre of a given song based on its audio features.

Dataset: The Million Song Dataset is a freely available collection of audio features and metadata for a million contemporary popular music tracks. The dataset is available on Kaggle and can be downloaded from the following link: https://www.kaggle.com/c/msdchallenge/data. The dataset contains a training set of 436,757 songs and a test set of 104,045 songs.

Framework:

1. **Data Loading:** The first step is to load the dataset into the code. The code uses pandas library to read the data files in CSV format.
2. **Data Exploration:** In this step, we explore the data by analyzing its different features and statistical summaries. We also visualize the data using histograms, scatter plots, and box plots.
3. **Data Preprocessing:** In this step, we preprocess the data by removing any missing values, scaling the features, and encoding the categorical variable (genre).
4. **Feature Selection:** In this step, we select the relevant features from the dataset that contribute the most to the classification task. We use the SelectKBest algorithm to select the top K features.
5. **Model Selection:** In this step, we select the appropriate model for our classification task. We use different models like Logistic Regression, Decision Tree, Random Forest, and KNN and compare their performance using accuracy, precision, recall, and F1-score metrics.
6. **Hyperparameter Tuning:** In this step, we tune the hyperparameters of the selected model to improve its performance. We use GridSearchCV to search the best hyperparameters.
7. **Model Evaluation:** In this step, we evaluate the performance of our final model on the test set using the accuracy, precision, recall, and F1-score metrics.

Conclusion: The code presented here provides a comprehensive framework for building a music genre classification model using the Million Song Dataset. With some

modifications and extensions, this code can be used to build a more advanced model that can be used in real-world music applications.

Requirements:

1. Python 3.7 or above
2. pandas, numpy, matplotlib, seaborn, scikit-learn libraries

To run the code, you can simply copy and paste it into a Python IDE (like Jupyter Notebook or Spyder) and run it line by line. You will need to install the required libraries before running the code.

After running the code, you will get the classification report of the final model, which contains the accuracy, precision, recall, and F1-score metrics. You can use these metrics to evaluate the performance of the model on the test set.

Future Work :

1. **Feature Engineering**: Feature engineering is a crucial step in any machine learning project. To improve the accuracy of our model, we can explore more features, such as song length, tempo, and the use of specific instruments. We can also try different feature scaling techniques and data normalization methods to see if they improve our model's performance.
2. **Model Selection**: In our current project, we have used a random forest classifier to predict music genres. However, there are several other machine learning algorithms that we can try to improve the accuracy of our model. For example, we can try using a support vector machine (SVM) or a neural network for this task.
3. **Hyperparameter Tuning**: Hyperparameter tuning is a crucial step in any machine learning project. We can use techniques like grid search or random search to find the optimal hyperparameters for our models.
4. **Ensemble Learning**: Ensemble learning is a technique where we combine multiple models to improve the accuracy of our predictions. We can try using different ensemble learning techniques such as bagging, boosting, and stacking to improve the performance of our model.
5. **Deploying the Model**: After we have built a robust and accurate model, we can deploy it to make predictions on new data. We can deploy the model as a web application or integrate it into an existing software system. We can also use the model for music recommendation systems or for personalized music streaming services.

Step-by-Step Guide to Implement Future Work

1. **Feature Engineering**: To perform feature engineering, we need to understand the dataset and explore different features that can improve the model's accuracy. We can try different techniques such as one-hot encoding, feature scaling, and data normalization. We can also experiment with different algorithms to extract features from the audio data.
2. **Model Selection**: To select the best machine learning algorithm for our project, we can try different algorithms such as SVM, neural networks, and ensemble methods. We can also use techniques such as cross-validation to evaluate the performance of each model.
3. **Hyperparameter Tuning**: Once we have selected the best algorithm, we can fine-tune its hyperparameters to improve its performance. We can use techniques such as grid search or random search to find the optimal hyperparameters for our model.
4. **Ensemble Learning**: After we have built several models, we can use ensemble learning techniques to combine their predictions and improve the overall accuracy of our model.
5. **Deploying the Model**: After we have built a robust and accurate model, we can deploy it as a web application or integrate it into an existing software system. We can use frameworks such as Flask or Django to build a web application that can make predictions

on new data. We can also use the model for music recommendation systems or for personalized music streaming services.

6. **Maintenance and Monitoring**: After deploying the model, we need to monitor its performance and ensure that it continues to make accurate predictions on new data. We also need to maintain the model and update it as new data becomes available or as the user requirements change. We can use techniques such as A/B testing and monitoring to evaluate the model's performance and identify areas for improvement.

Exercise :

Try to answers the following questions by yourself to check your understanding for this project. If stuck, detailed answers for the questions are also provided.

1. **What is the purpose of the extract_features() function in this code, and how does it work?**
   Answer: The **extract_features()** function is used to extract various features from the audio data that will be used for modeling. It works by using the **librosa** library to load the audio file and then extracting various features such as the mel frequency cepstral coefficients (MFCCs), spectral centroid, and chroma features.

2. **What is the purpose of the train_model() function, and how does it work?**
   Answer: The **train_model()** function is used to train the machine learning model to predict the genre of a given audio file. It works by first splitting the data into training and validation sets, then fitting a support vector machine (SVM) classifier to the training data. The hyperparameters of the SVM are tuned using grid search and cross-validation to optimize performance.

3. **What is the purpose of the plot_confusion_matrix() function, and how does it work?**
   Answer: The **plot_confusion_matrix()** function is used to plot the confusion matrix of the model's predictions on the test data. It works by using the **sklearn** library to compute the confusion matrix, and then plotting the matrix using the **matplotlib** library.

4. **What are some potential limitations of this model, and how could they be addressed in future work?**
   Answer: Some potential limitations of this model include the limited size and diversity of the dataset used for training, as well as the reliance on hand-crafted features. Future work could address these limitations by collecting a larger and more diverse dataset, and exploring the use of deep learning models that can learn features directly from the audio data.

5. **What is the purpose of the predict_genre() function, and how can it be used?**
   Answer: The **predict_genre()** function is used to predict the genre of a given audio file using the trained machine learning model. It works by first extracting the features from the audio file using the **extract_features()** function, and then passing the features

through the trained SVM classifier. This function can be used to predict the genre of any new audio file that is passed to it as input.